# Factory Control Unit: modellgetriebene Hard- und Software mit 3D-Web-Oberfläche für die Lernfabrik „Factory in a Box“

**Klaus Liebler** · [Hochschule, Fachbereich] · klaus@klaus-liebler.de

## 1 Problem, Kontext und eigener Beitrag

**Problem und Ziel.** Kommerzielle Lernfabriken wie die fischertechnik Training Factory [1] bringen eine feste Steuerung mit; Eigenbauten bieten selten industrieübliche Schnittstellen. „Factory in a Box“ ist eine kompakte Lernfabrik (u. a. Förderband, Roboterarm Waveshare RoArm-M3, pneumatische Druckregelstrecke; Materialbudget rund 600 €). Ziel ist ihre Steuerung, die **Factory Control Unit (FCU)**: eine Platine mit Firmware, die alle Sensoren und Aktoren bedient und über Modbus, OPC UA und eine Browser-Oberfläche zugänglich macht, wobei das Prozessabbild nur einmal beschrieben werden soll. **Kontext.** OpenPLC [2] bietet eine quelloffene SPS-Laufzeit; die FCU ist dagegen ein Ein-/Ausgabegerät, dessen Protokollstack und Prozessabbild selbst Lerngegenstand sind. Als Schnittstellen dienen Modbus [3] und OPC UA [4]. **Beitrag.** (a) Eine Steuerplatine auf Basis des STM32H563; (b) ein modellgetriebener Ansatz, bei dem ein Informationsmodell C++-Code für Modbus und OPC UA sowie einen Teil des Web-UI erzeugt; (c) ein Web-UI mit eigener 3D-Engine, digitalem Schatten des Roboterarms und grafischem Teaching; (d) eine Regelungstechnik-Lehranwendung.

## 2 Vorgehen und Methodik

**Hardware und Firmware.** Die Platine vereint einen STM32H563 (Cortex-M33) mit Ethernet, USB-C (PD-Sink) und FDCAN, Schrittmotortreiber (TMC2209), Wägezelle, I²C-Sensoren, Leistungsmonitor INA226, Lichtschranken, Ventil- und PWM-Treiber, Bus-Servo-Anschluss für den Roboterarm und einen EtherCAT-Slave-Controller (LAN9253). Die C++-Firmware läuft auf Eclipse ThreadX mit NetX Duo und USBX; ein E/A-Thread (50 ms) pflegt das zentrale Registermodell, das Modbus TCP, Modbus RTU (USB), ein schlanker OPC-UA-Server (Eigenentwicklung, Basic256Sha256) und HTTPS/WebSocket gemeinsam nutzen.

**Informationsmodell und Codegenerierung.** Das Prozessabbild ist einmalig als annotierte C#-Klassen beschrieben (Abb. 1): 16 Klassen bilden Regionen, 75 Felder Register; Attribute legen Zugriffsrecht, Einheit, Adresse, Pin, Wertebereich und Darstellungsart fest. Ein Generator (UniversalRegisterAccess), vom .NET-Build-Orchestrator aufgerufen, erzeugt daraus (1) C++-Header mit Adresskonstanten, Bitmasken und typisierten Accessoren für Modbus, (2) eine constexpr-Knotentabelle des OPC-UA-Adressraums (Objekte je Region, Variablen mit Datentyp, Zugriffsrechten und Callbacks; 32-Bit-Werte aus Registerpaaren), die ohne Laufzeitaufbau im Flash liegt, und (3) eine TypeScript-Registerkarte, aus der das Web-UI seine generischen Register-Panels (Tabellen, Schalter, Schieberegler mit Grenzen aus dem Schema) rendert. Schemaänderungen wirken so gleichzeitig auf Firmware und Oberfläche; Inkonsistenzen zwischen Modbus, OPC UA und UI werden zur Übersetzungszeit vermieden. Ein zweites Schema (BestBinaryBuffers: rund 30 Nachrichten, Union-Typen für Missionsschritte) erzeugt Kodierer, Dekodierer und maximale Puffergrößen des binären WebSocket-Protokolls in C++ und TypeScript (Tab. 1).

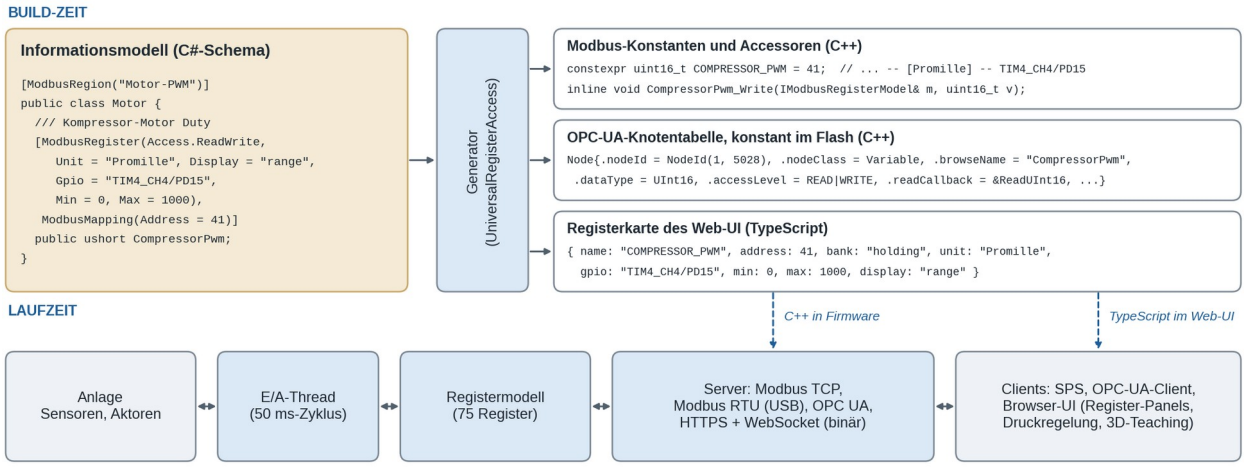



**Abb. 1:** Ein Informationsmodell erzeugt zur Build-Zeit Modbus-, OPC-UA- und Web-UI-Code; zur Laufzeit teilen alle Zugriffswege ein Registermodell.

**Tab. 1:** Aus dem Registerschema (erste drei Zeilen) und dem Nachrichtenschema (letzte Zeile) erzeugte Dateien.

| Artefakt | Sprache | Inhalt | Zeilen |
|---|---|---|---|
| `modbus_registers_generated.hh` | C++ | Adresskonstanten, Bitmasken, typisierte Accessoren | 551 |
| `opcua_registers_generated.hh` | C++ | OPC-UA-Adressraum als constexpr-Knotentabelle (>110 Knoten) | 902 |
| `register-map.ts` | TS | Registermetadaten für Register-Panels und Regelstrecke | 186 |
| `ws_protocol.hh / ws-protocol.ts` | C++/TS | 30 WebSocket-Nachrichten: Kodierer, Dekodierer, Puffer | 2586 / 1333 |

**Web-UI.** Das Web-UI ist eine Single-Page-Anwendung aus Lit-Web-Components (TypeScript, Vite; rund 5800 Zeilen in den Anwendungsseiten). Der Build erzeugt eine einzige Brotli-komprimierte Datei (84 kB statt 460 kB), die per HTTPS aus dem Flash ausgeliefert wird; alle Anwendungsdaten laufen über einen binären WebSocket. Seiten sind generische Register-Panels, Power Management (INA226), eine System- und Task-Manager-Ansicht mit ThreadX-Live-Statistik, Roboter-Teach und die Druckregelstrecke. Letztere findet ihre Register über den Namen in der generierten Karte und führt im Browser Kennlinienaufnahme, Sprungantwort (Totzeit, 63-%-Zeitkonstante) und Reglerbetrieb (P/PI/PID, 500-ms-Takt, Entwurf nach T-Summen-Regel oder Chien–Hrones–Reswick [5], Anti-Windup) aus; die Firmware bleibt ein reines Register-Interface.

**3D-Engine und Roboterdarstellung.** Auf der schlanken WebGL-Schicht OGL läuft ein eigener PBR-Shader (Cook-Torrance-GGX, analytischer Himmelsverlauf statt Cubemap, daher ohne Bild-Assets). Die Netze des Roboters werden offline aus den URDF-/STL-Daten des Herstellers per Skript (Vertex-Verschweißung, optionale Vereinfachung) in ein TypeScript-Modul überführt (Abb. 2); Vakuumsauger und Werkstück sind prozedurale Primitive. Vektor-/Quaternionenmathematik, Canvas-Overlay mit Bodenraster und selbst implementiertem Verschiebe-/Drehgizmo samt Hit-Testing, SVG-ViewCube und Orbit-/Pan-/Zoom-Kamera sind eigener Code (rund 1500 Zeilen TypeScript). Eine geschlossene (nicht iterative) inverse Kinematik des 3R-Arms mit Basisdrehung wählt aus vier Konfigurationen die der aktuellen Pose nächste (an 2000 Zufallsposen geprüft); das Raster färbt sich bei Gelenkgrenzen oder unerreichbarem Ziel gelb bzw. rot.

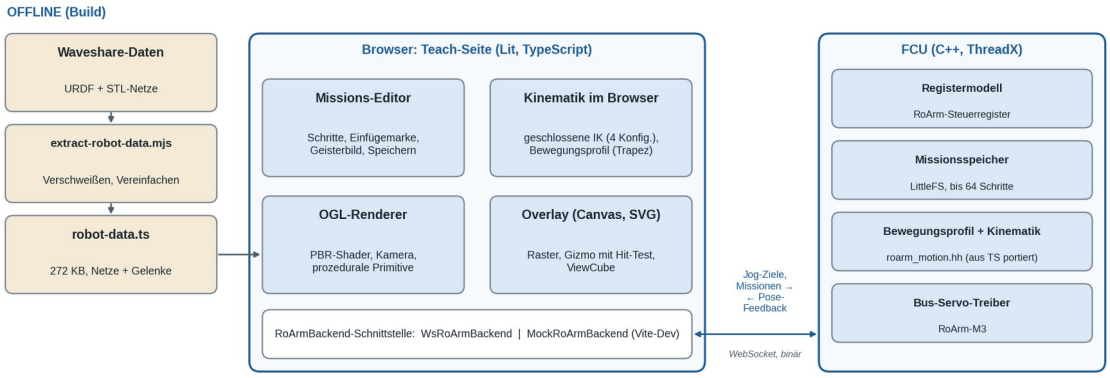



**Abb. 2:** Teach-Seite: Roboterdaten aus der Offline-Werkzeugkette, Browser-Komponenten und Kopplung zur FCU über eine Backend-Schnittstelle.

**Teaching und Missionen.** Zielposen entstehen über Gelenkschieber, kartesische Werte oder den Gizmo (Abb. 3); die FCU erhält Jog-Ziele und meldet Gelenkwinkel, Position und Servostatus zurück, womit das 3D-Modell als digitaler Schatten der realen Pose folgt. Missionen bestehen aus Gelenkfahrten (exakte Pose oder Stützstelle), Vakuum-Schaltschritten und Wartezeiten; sie werden an einer Einfügemarke bearbeitet (Verschieben, Löschen, Geisterbild der vorherigen Pose), als benannte Slots mit bis zu 64 Schritten im LittleFS gespeichert und per Register auch von Modbus- oder OPC-UA-Clients gestartet; ein aktiver Teach-Modus sperrt den Missionsstart. Das trapezförmige Geschwindigkeitsprofil und die Kinematik der Waveshare-Referenz wurden zunächst in TypeScript entwickelt und identisch nach C++ portiert; sie sind noch nicht an realer Hardware verifiziert. Ein Mock-Backend erlaubt die UI-Entwicklung ohne Hardware; Abb. 3 stammt aus diesem Aufbau.

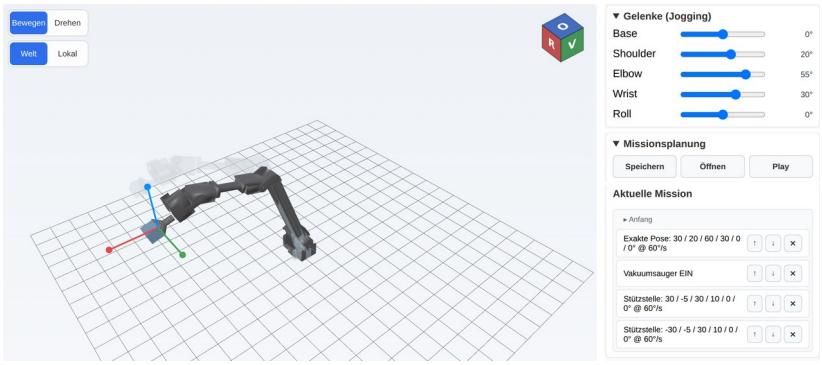



**Abb. 3:** Teach-Seite (Ausschnitte): 3D-Modell mit Gizmo, ViewCube und Geisterbild; Gelenkschieber und Missionsliste (Entwicklungsserver, Mock-Backend).

## 3 Ergebnisse und Reifegrad

Die grundsätzliche Funktion aller Ein-/Ausgänge und Kommunikationsschnittstellen wurde im Rahmen einer Bachelorarbeit geprüft; die EtherCAT-Schnittstelle wird in Kürze funktionsfähig sein. Codegenerator, Web-UI, 3D-Teaching und Druckregelstrecke sind implementiert. Reifegrad: Funktionsmuster im Labor (etwa TRL 4). Offen sind der Einsatz in einer Lehrveranstaltung, Zeitmessungen und die Verifikation der Roboterkinematik an realer Hardware.

## 4 Nutzen für Lehre, Praxis und Forschung

In der **Lehre** greifen SPS-Übungen über Modbus TCP oder OPC UA auf reale Ein- und Ausgänge zu, die Regelungstechnik nutzt die Druckregelstrecke, Robotikprojekte die Teach-Oberfläche; Abschlussarbeiten erweitern Hard- und Software. Der INA226 liefert Spannung, Strom und Leistung der Anlage für **Energiemonitoring**. Für **Praxis und Forschung** zeigt der Ansatz konsistente Feldbus-, OPC-UA- und Web-Sichten aus einem Modell auf einem Mikrocontroller.

## Literatur


[1] fischertechnik GmbH: Training Factory Industry 4.0 24V – Accompanying booklet, Stand 22.03.2024.

[2] Alves, T. et al.: OpenPLC: An open source alternative to automation. IEEE Global Humanitarian Technology Conference (GHTC), 2014.

[3] Modbus Organization: MODBUS Application Protocol Specification V1.1b3, 2012.

[4] IEC 62541: OPC Unified Architecture, Teil 6: Mappings.

[5] Chien, K. L.; Hrones, J. A.; Reswick, J. B.: On the automatic control of generalized passive systems. Trans. ASME 74 (1952), S. 175–185.